

Week 3: MSA 운영 관점과 서비스 연결

Overview

3주차는 Week 2에서 만든 단일 Docker/Compose 실행 경험을 여러 서비스가 연결된 MSA(Microservice Architecture) 운영 문제로 확장한다. 학생은 MSA를 개발 방법론으로 외우지 않고, 인프라/DevOps 엔지니어가 실행하고 관찰하고 장애를 줄여야 하는 서비스 토폴로지로 다룬다.

이번 주의 중심 질문은 다음과 같다.

서비스가 frontend, api, worker, database로 나뉘면 실행, 연결, 설정, 로그, 장애 대응, 배포 협업은 어떻게 달라지는가?

MSA는 항상 더 좋은 정답이 아니다. 서비스 경계가 명확하고 팀/배포/확장 요구가 있을 때 장점이 생기지만, network hop, latency, 로그 분산, 장애 전파, 설정 관리, 배포 순서 같은 운영 비용도 늘어난다. 이번 주의 평가는 MSA 찬반이 아니라 서비스 경계와 운영 증거를 설명하는 능력을 기준으로 한다.

Learning Goals

- Monolith와 MSA의 차이를 배포 단위, 장애 영향 범위, 데이터 책임, 운영 복잡도 관점으로 설명한다.
- 서비스 목록, 포트, 프로토콜, 환경변수, 의존 서비스, health endpoint, 로그 위치를 운영 문서로 정리한다.
- Docker Compose로 frontend, api, worker, database를 실행하고 서비스별 상태를 확인한다.
- frontend-api, api-database, worker-api 연결 문제를 network와 configuration 관점에서 분석한다.
- health check, timeout, retry, graceful degradation의 필요성을 장애 전파 관점으로 설명한다.
- 분산 로그와 request id/correlation id가 필요한 이유를 개발팀 협업 요청으로 정리한다.
- Compose 수동 운영의 한계를 Kubernetes의 Pod, Deployment, Service, ConfigMap, Secret 필요성과 연결한다.

Weekly Keywords

- MSA
- monolith
- service boundary
- topology
- frontend
- API
- worker
- database
- dependency map
- service discovery
- health check
- liveness
- readiness
- timeout
- retry
- graceful degradation
- correlation id
- request id
- blast radius
- Docker Compose
- Kubernetes readiness

Schedule Index

- Day 1: 2주차 복습, Monolith vs MSA, 서비스 토폴로지, 표준 MSA 실습 앱 실행
- Day 2: 서비스 간 HTTP 통신, frontend-api 연결, api-database 연결, health check, 연결 실패 분석
- Day 3: 데이터 의존성, worker/queue 개념, 장애 전파, timeout/retry, 분산 로그와 관찰 가능성
- Day 4: 설정 관리, dev/prod Compose 분리, API 계약, 버전/배포 협업, MSA 운영 비용, Kubernetes 필요성

- Day 5: 통합 실습, 설정 변경, 재빌드/재실행, 장애 주입/복구, 발표, Kubernetes 연결

Week 2 To Week 3 Mapping

Week 2 Docker/Compose evidence	Week 3 MSA 표현	운영 판단
하나의 service	여러 service topology	어떤 서비스가 누구에게 의존하는가
port binding	내부 port와 외부 endpoint	사용자는 어디로 들어오고 서비스끼리는 어디로 통신하는가
compose.yaml	topology/runbook source	실행 순서와 설정이 파일에 남아 있는가
container logs	service별 distributed logs	요청 흐름을 여러 로그에서 이어 볼 수 있는가
environment variable	service별 config contract	설정 누락이 어느 서비스 장애로 나타나는가
volume/database	data ownership	여러 서비스가 DB를 공유할 때 어떤 위험이 생기는가
health check preview	readiness/liveness	서비스가 실행 중인지와 요청 받을 준비가 됐는지 구분하는가
Docker readiness	Kubernetes readiness	수동 Compose 운영의 한계를 설명할 수 있는가

Required Deliverables

- MSA 실습 애플리케이션 실행 가능한 `compose.yaml`
- frontend, api, worker, database 요청 흐름과 의존성 다이어그램
- 서비스별 실행 조건 표: image/build, port, environment variable, dependency, health check, log command
- 배포 설정 또는 환경 설정 변경 기록 1개
- 장애 주입 및 복구 기록 1개
- 개발팀에 전달할 장애 리포트 또는 운영 개선 요청서 1개
- 인프라 관점에서 본 MSA 장점과 운영 비용 회고
- 4주차 Kubernetes readiness checklist

Practice Environment

항목	기준
Docker	docker version, docker compose version 실행 가능
Local ports	frontend host port 18083, optional api debug port 18084 사용 가능
Repository	week3/day1/labs/msa-demo 실습 앱 실행 가능
Browser/curl	frontend 접속, api health, service status 확인 가능
Cost	Week 3는 로컬 Docker 중심이다. cloud resource를 만들지 않는다.
Security	.env, credential, token, DB password를 public screenshot/README에 그대로 남기지 않는다.

Official References

Topic	Reference	확인할 키워드
Microservices on AWS	https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/microservices.html	service, API, database, deployment
AWS Microservices	https://aws.amazon.com/microservices/	independent component, business

		capability
Martin Fowler Microservices Guide	https://martinfowler.com/microservices/	independently deployable, lightweight communication
Docker Compose	https://docs.docker.com/compose/	services, networks, depends_on, healthcheck
Compose services reference	https://docs.docker.com/reference/compose-file/services/	healthcheck, depends_on, environment
Google SRE Cascading Failures	https://sre.google/sre-book/addressing-cascading-failures/	failure propagation, overload, mitigation
OpenTelemetry Concepts	https://opentelemetry.io/docs/concepts/	traces, metrics, logs, observability
Twelve-Factor App	https://12factor.net/	config, backing services, logs

Cost And Security Notes

- Week 3 실습은 로컬 Docker Compose 중심이므로 AWS, Kubernetes cluster, 유료 DB를 만들지 않는다.
- MSA는 서비스 수가 늘면서 로그, 네트워크, 이미지, 컨테이너 수가 늘어난다. 로컬에서도 disk/memory 사용량을 확인하고 종료 후 정리한다.
- 실습 DB password는 예시값이어도 공개 repository에 그대로 올리지 않는다. 필요한 것은 값이 아니라 환경변수 이름과 관리 기준이다.
- 외부 API key, token, cloud credential을 MSA 실습 앱에 넣지 않는다.

Weekly Checklist

- ☐ 표준 MSA 실습 앱을 `docker compose up --build` 로 실행했다.
- ☐ frontend, api, worker, database 역할을 설명했다.
- ☐ service name DNS와 localhost의 차이를 설명했다.
- ☐ `/health` 또는 상태 endpoint로 service readiness를 확인했다.
- ☐ 잘못된 API URL, DB host, 환경변수 중 하나의 장애를 재현하고 복구했다.
- ☐ service별 로그에서 요청 흐름을 추적했다.
- ☐ timeout/retry가 필요한 이유와 위험을 모두 설명했다.
- ☐ MSA를 Kubernetes로 옮길 때 필요한 리소스 후보를 적었다.

Glossary

3주차 용어는 [glossary.md](#)를 기준으로 정리한다.

Next Week Connection

4주차 Kubernetes는 3주차 MSA 운영 문제를 선언적 리소스와 controller로 다루는 주간이다. Week 3에서 확인한 frontend, api, worker, database, health check, config, secret, service discovery는 Week 4의 Pod, Deployment, Service, ConfigMap, Secret, Ingress로 확장된다.